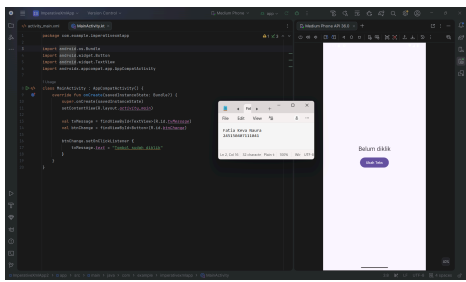
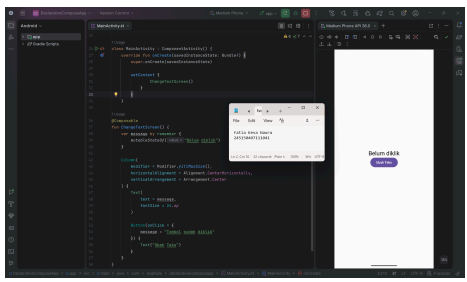
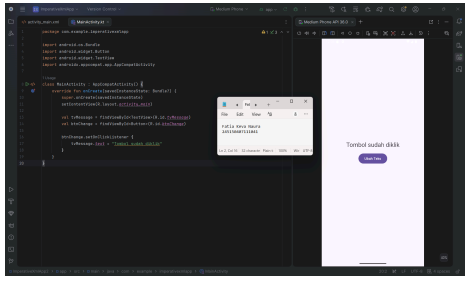
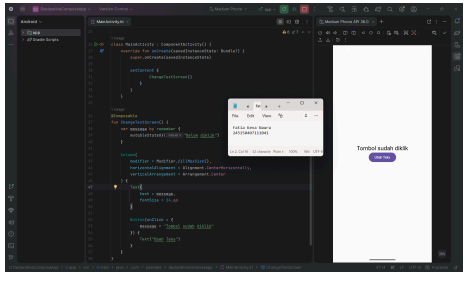


Nama: Fatia Keva Naura
NIM: 245150407111041

Modul 3 (Event, State, dan Implisit Intent pada Jetpack Compose)

-> Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

	XML	Compose
Sebelum	 Screenshot of an Android Studio interface showing an XML layout file. The text in the TextView is "Belum diklik". A small dialog box is visible over the code editor.	 Screenshot of an Android Studio interface showing a Compose layout file. The text in the TextView is "Belum diklik". A small dialog box is visible over the code editor.
Sesudah	 Screenshot of an Android Studio interface showing an XML layout file. The text in the TextView is "Tombol sudah diklik". A small dialog box is visible over the code editor.	 Screenshot of an Android Studio interface showing a Compose layout file. The text in the TextView is "Tombol sudah diklik". A small dialog box is visible over the code editor.

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.
= Pada project XML/View, objek yang diubah secara langsung adalah TextView dengan id tvMessage. Ketika tombol "Ubah Teks" diklik, program langsung mengubah nilai text pada TextView tersebut.
Baris kodenya:

```
btnChange.setOnClickListener {  
    tvMessage.text = "Tombol sudah diklik"  
}
```

Jadi, alurnya adalah tombol diklik -> TextView diambil -> teks TextView diubah secara langsung. Ini merupakan pendekatan imperatif, karena program memberikan perintah langsung kepada komponen UI yang ingin diubah.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

= Pada project Jetpack Compose, nilai yang diubah ketika tombol diklik adalah state message. State tersebut awalnya berisi "Belum diklik", kemudian ketika tombol diklik nilainya berubah menjadi "Tombol sudah diklik".

Baris kodenya:

```
var message by remember {  
    mutableStateOf("Belum diklik")  
}
```

Kemudian pada tombol:

```
Button(onClick = {  
    message = "Tombol sudah diklik"  
}) {  
    Text("Ubah Teks")  
}
```

Jadi, yang diubah bukan Text secara langsung, tetapi nilai state message. Setelah state berubah, Compose akan melakukan recomposition sehingga teks yang ditampilkan ikut berubah.

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

= Karena Jetpack Compose menggunakan pendekatan deklaratif. Pada Compose, kita tidak perlu mengubah komponen UI secara langsung seperti pada XML/View. Kita cukup mengubah state yang menjadi sumber data tampilan. Ketika state message berubah, Compose akan mendeteksi perubahan tersebut dan melakukan recomposition, sehingga Text akan menampilkan nilai state yang terbaru.

Jadi, pada Compose alurnya adalah: event -> state berubah -> recomposition -> tampilan berubah. Sedangkan pada XML/View: event -> View -> properti View diubah.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

= Pada XML/View, ketika pengguna menekan tombol "Ubah Teks", terjadi sebuah event berupa klik tombol. Event tersebut ditangani menggunakan setOnClickListener. Setelah itu program mengakses View, yaitu TextView dengan id tvMessage. Kemudian program mengubah properti text pada TextView menjadi "Tombol sudah diklik".

Jadi alurnya: Event (tombol diklik) -> View (tvMessage) -> perubahan tampilan (teks menjadi "Tombol sudah diklik").

Pendekatan ini disebut imperatif karena program secara langsung memberikan perintah untuk mengubah komponen UI.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

= Pada Jetpack Compose, ketika pengguna menekan tombol "Ubah Teks", terjadi event berupa klik tombol. Event tersebut kemudian mengubah nilai state message dari "Belum diklik" menjadi "Tombol sudah diklik".

Karena state berubah, Compose melakukan recomposition pada bagian UI yang menggunakan state tersebut. Setelah itu, Text menampilkan nilai message yang baru sehingga tampilan berubah menjadi "Tombol sudah diklik".

Jadi alurnya: Event (tombol diklik) -> state message berubah -> recomposition -> tampilan Text berubah.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: var message = "Belum di klik"? Jelaskan penyebabnya.

= Jika message diganti menjadi variabel biasa:

```
var message = "Belum diklik"
```

maka perubahan nilai message tidak otomatis membuat tampilan UI berubah ketika tombol diklik. Hal ini terjadi karena variabel biasa bukan merupakan state yang dapat diamati oleh Compose. Compose tidak mengetahui bahwa variabel tersebut berubah sehingga tidak melakukan recomposition untuk memperbarui Text. Agar perubahan nilai dapat memicu pembaruan UI, message perlu dibuat sebagai state, misalnya:

```
var message by remember { mutableStateOf("Belum diklik") }
```

Dengan begitu, ketika message berubah, Compose dapat melakukan recomposition.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

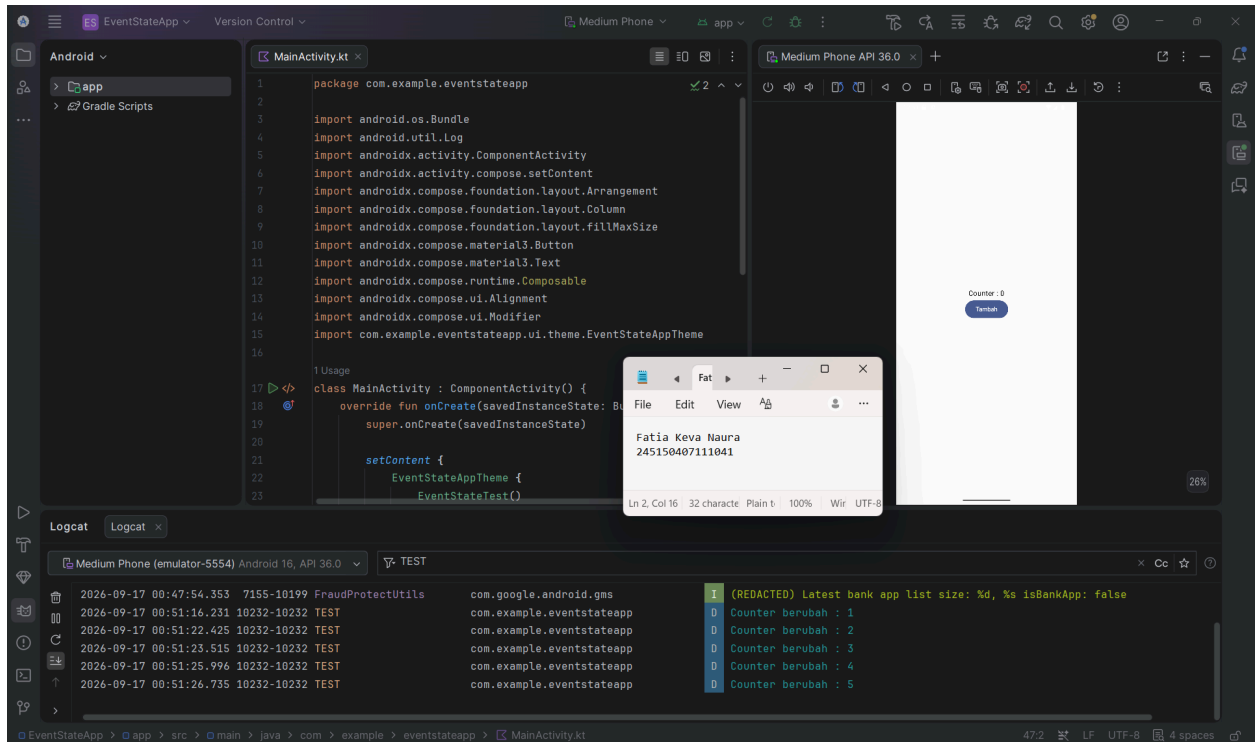
= Menurut saya, perbedaan utama antara imperatif dan deklaratif adalah cara program mengatur perubahan tampilan. Pada pendekatan imperatif, program memberikan perintah secara langsung kepada komponen UI yang ingin diubah. Contohnya adalah mencari TextView, kemudian mengubah text-nya ketika tombol diklik.

Sedangkan pada pendekatan deklaratif seperti Jetpack Compose, program lebih berfokus pada kondisi atau state yang menentukan tampilan UI. Ketika state berubah, Compose akan melakukan recomposition dan memperbarui tampilan yang berkaitan dengan state tersebut.

Jadi, kesimpulannya imperatif berarti memberi tahu bagaimana UI harus diubah, sedangkan deklaratif berarti menentukan seperti apa UI berdasarkan state saat ini.

-> Event Handling State

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali



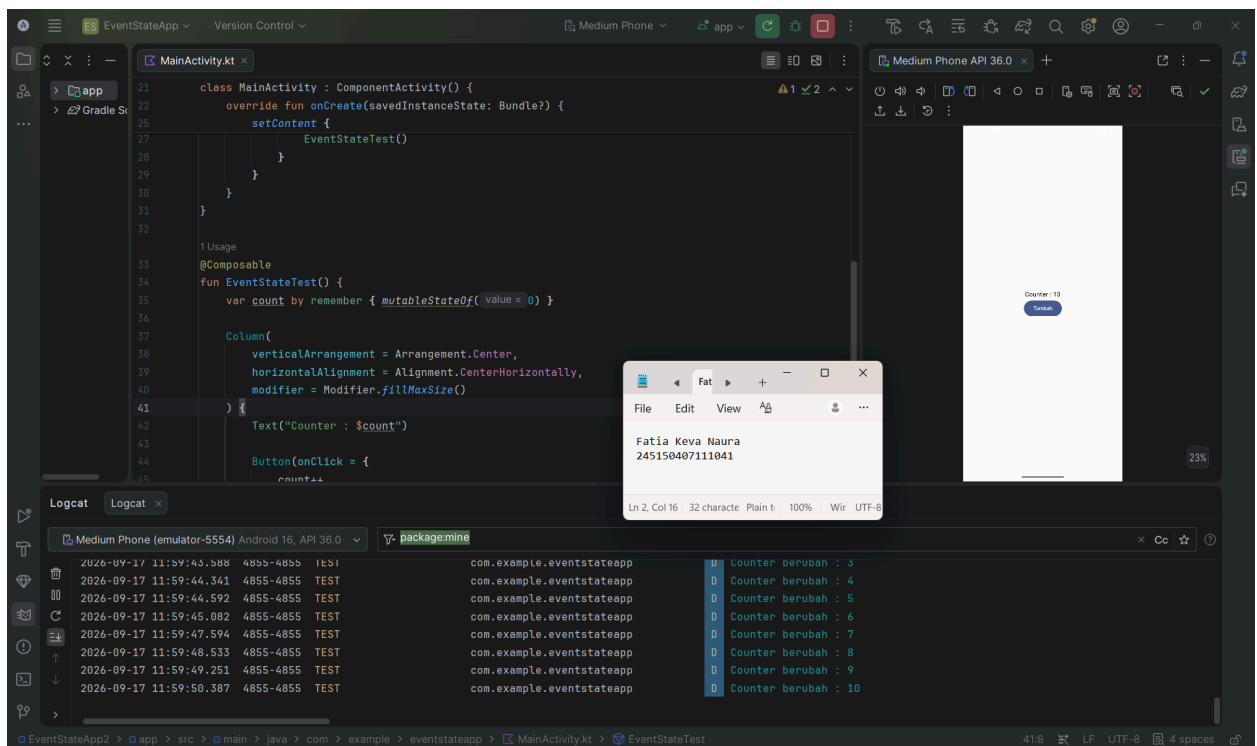
2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
= Setiap kali tombol "Tambah" diklik, LogCat menampilkan pesan dengan tag TEST dan nilai counter bertambah. Pada percobaan saya, setelah tombol diklik sebanyak lima kali, muncul:
Counter berubah : 1
Counter berubah : 2
Counter berubah : 3
Counter berubah : 4
Counter berubah : 5
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
= Pada tampilan aplikasi, tulisan Counter : 0 tetap tidak berubah walaupun tombol "Tambah" sudah diklik sebanyak lima kali. Jadi, nilai yang ditampilkan pada UI tetap 0.
4. Apa yang menyebabkan hal tersebut terjadi?
= Hal tersebut terjadi karena variabel count masih berupa variabel biasa dengan kode:

```
var count = 0
```

Walaupun nilai count bertambah ketika tombol diklik, perubahan tersebut tidak dianggap sebagai state yang diamati oleh Jetpack Compose. Akibatnya, Compose tidak melakukan recomposition, sehingga tampilan Counter : 0 tidak diperbarui. Modul menjelaskan bahwa perubahan variabel biasa tidak otomatis memperbarui UI, sedangkan state seperti `mutableStateOf` dapat memicu recomposition.

-> UBAH KODE PROGRAM

8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
= Setiap kali tombol "Tambah" diklik, LogCat menampilkan pesan dengan tag TEST dan nilai counter yang bertambah. Pada percobaan saya, ketika tombol diklik, muncul pesan seperti Counter berubah : 1, kemudian Counter berubah : 2, dan seterusnya.
10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
= Tampilan aplikasi ikut berubah setiap kali tombol "Tambah" diklik. Nilai pada tulisan Counter bertambah sesuai dengan jumlah klik, misalnya dari Counter : 0 menjadi Counter : 1, kemudian Counter : 2, dan seterusnya.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

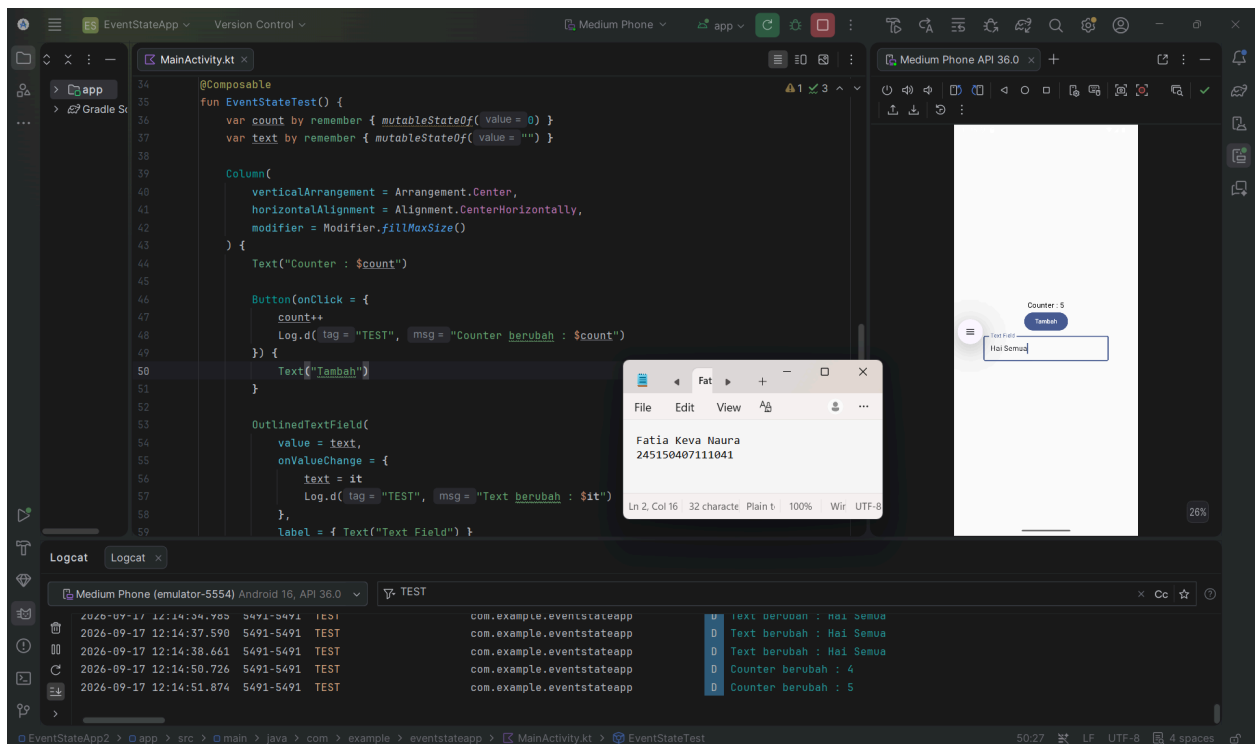
= Hal tersebut terjadi karena pada percobaan ini variabel count sudah menggunakan state dengan `remember { mutableStateOf(0) }`. Ketika nilai count berubah, Compose mengetahui adanya perubahan state dan melakukan recomposition sehingga tampilan UI ikut diperbarui. Berbeda dengan percobaan sebelumnya yang menggunakan variabel biasa `var count = 0`, perubahan nilainya tidak membuat UI diperbarui.

12. Apa yang dimaksud recomposition pada Compose?

= Recomposition adalah proses ketika Compose menjalankan kembali bagian UI yang bergantung pada state yang mengalami perubahan. Dengan adanya recomposition, tampilan UI dapat diperbarui sesuai dengan nilai state yang terbaru. Pada percobaan ini, ketika count berubah setelah tombol diklik, Compose melakukan recomposition sehingga nilai Counter pada layar ikut berubah.

-> UBAH KODE PROGRAM

14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

= Setiap kali menginputkan huruf pada Text Field, LogCat menampilkan pesan dengan tag TEST berupa Text berubah : diikuti oleh teks yang sedang dimasukkan. Pada percobaan saya, ketika mengetik teks "Hai Semua", LogCat menampilkan perubahan teks tersebut.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

= Setiap kali menginputkan huruf pada Text Field, tampilan Text Field akan langsung menampilkan karakter yang diketik. Teks yang ditampilkan akan bertambah sesuai dengan input yang diberikan, misalnya dari H, kemudian Ha, Hai, sampai menjadi Hai Semua.

17. Apa yang menyebabkan hal tersebut terjadi?

= Hal tersebut terjadi karena Text Field menggunakan state text dan event `onValueChange`. Ketika pengguna mengetik, event `onValueChange` dijalankan dan nilai text diperbarui menggunakan `text = it`. Karena text merupakan state, perubahan tersebut menyebabkan Compose melakukan recomposition sehingga tampilan Text Field ikut diperbarui.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

= State yang terdapat pada program terakhir adalah:

1. `count` dengan tipe data Integer (`Int`), digunakan untuk menyimpan jumlah klik tombol Tambah.
2. `text` dengan tipe data String, digunakan untuk menyimpan teks yang dimasukkan ke dalam Text Field.

Kodenya:

```
var count by remember { mutableStateOf(0) }  
var text by remember { mutableStateOf("") }
```

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

= Event yang terdapat pada program terakhir ada dua, yaitu:

1. `onClick`, yaitu event ketika tombol Tambah ditekan.
2. `onValueChange`, yaitu event ketika pengguna memasukkan atau mengubah teks pada Text Field.

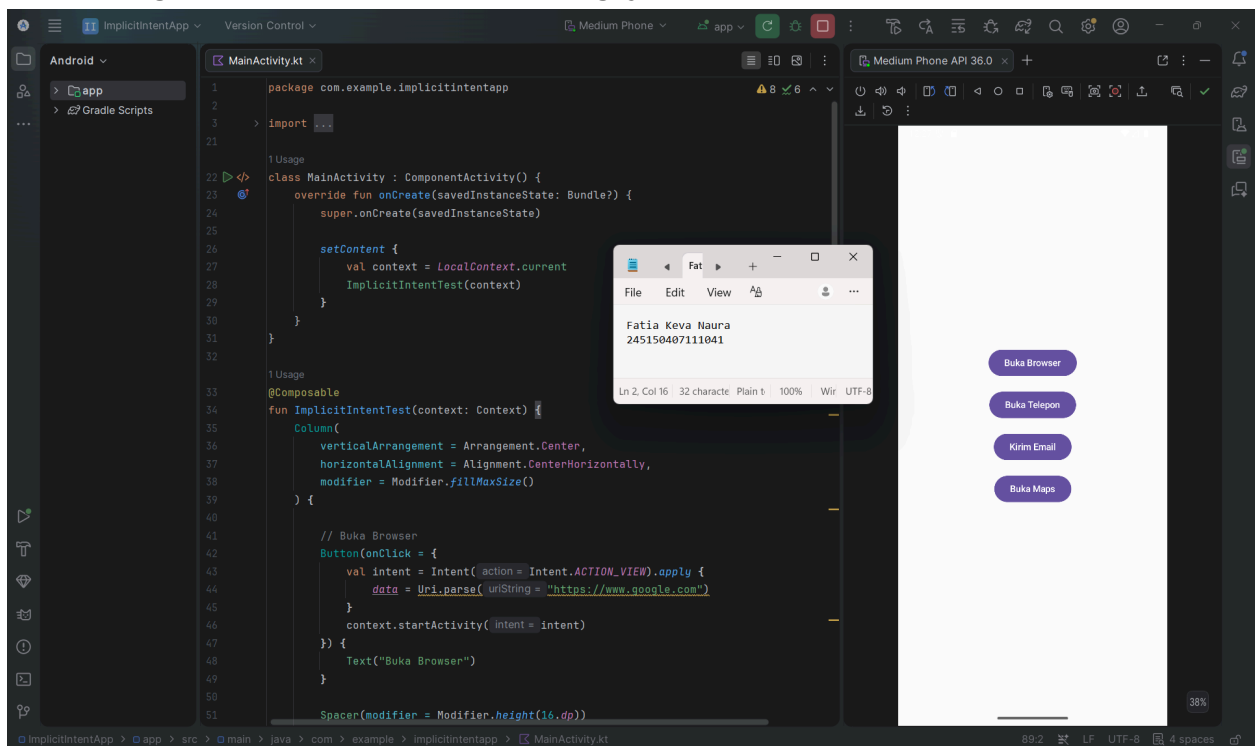
20. Kapan event - event tersebut dijalankan?

= Event `onClick` dijalankan ketika pengguna menekan tombol Tambah. Event tersebut akan menambah nilai `count` sebanyak satu.

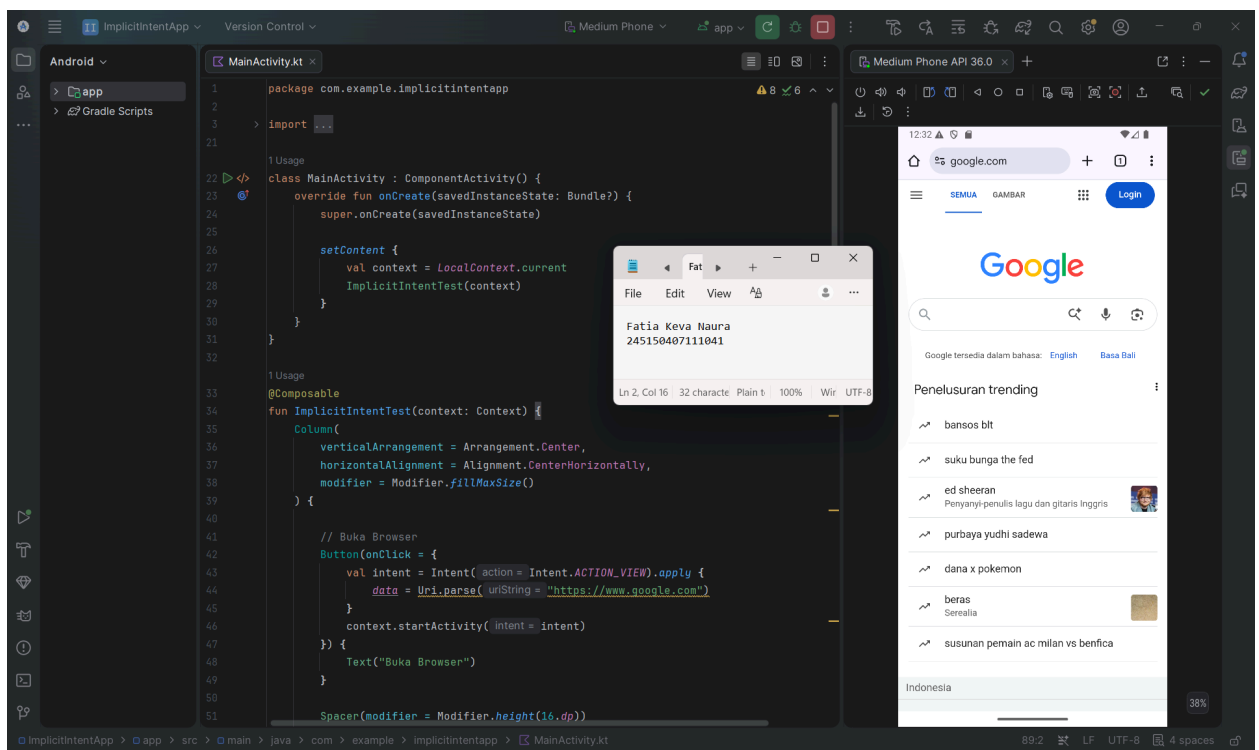
Sedangkan event `onValueChange` dijalankan ketika pengguna memasukkan atau mengubah karakter pada Text Field. Setiap perubahan input akan memperbarui nilai `text` dan mencatat perubahan tersebut ke LogCat.

-> Implisit Intent

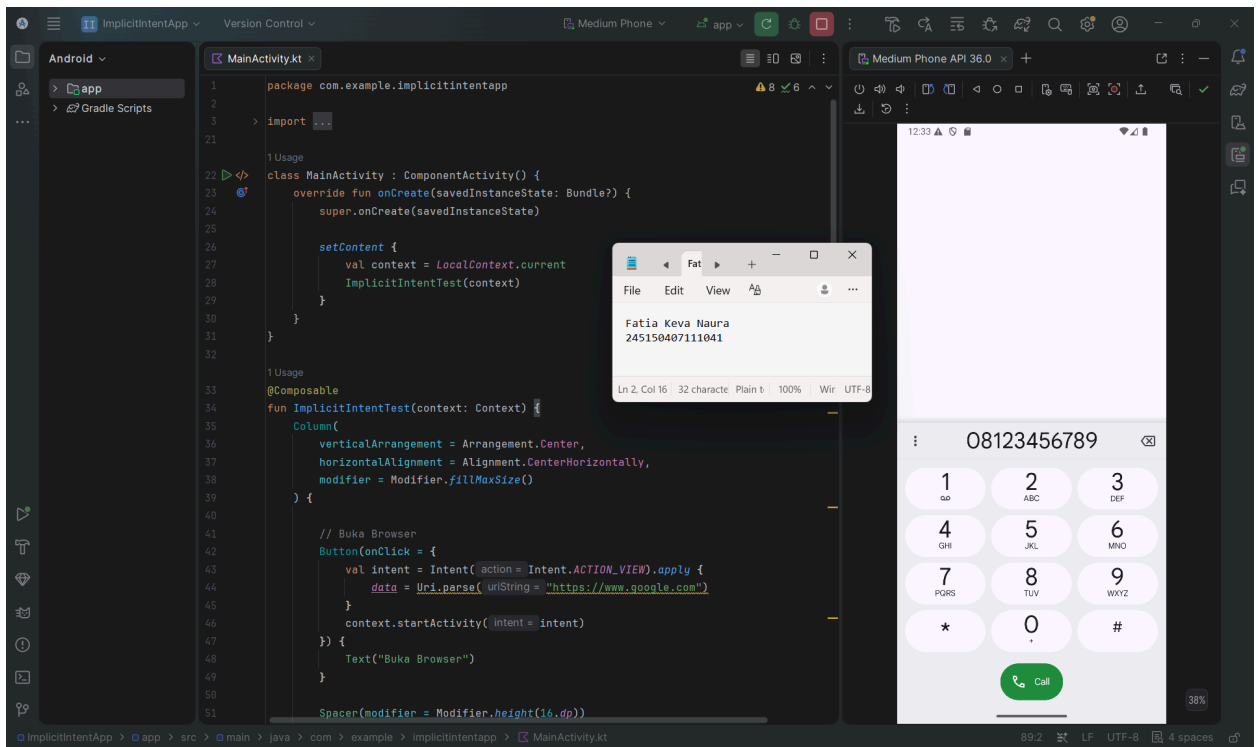
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:



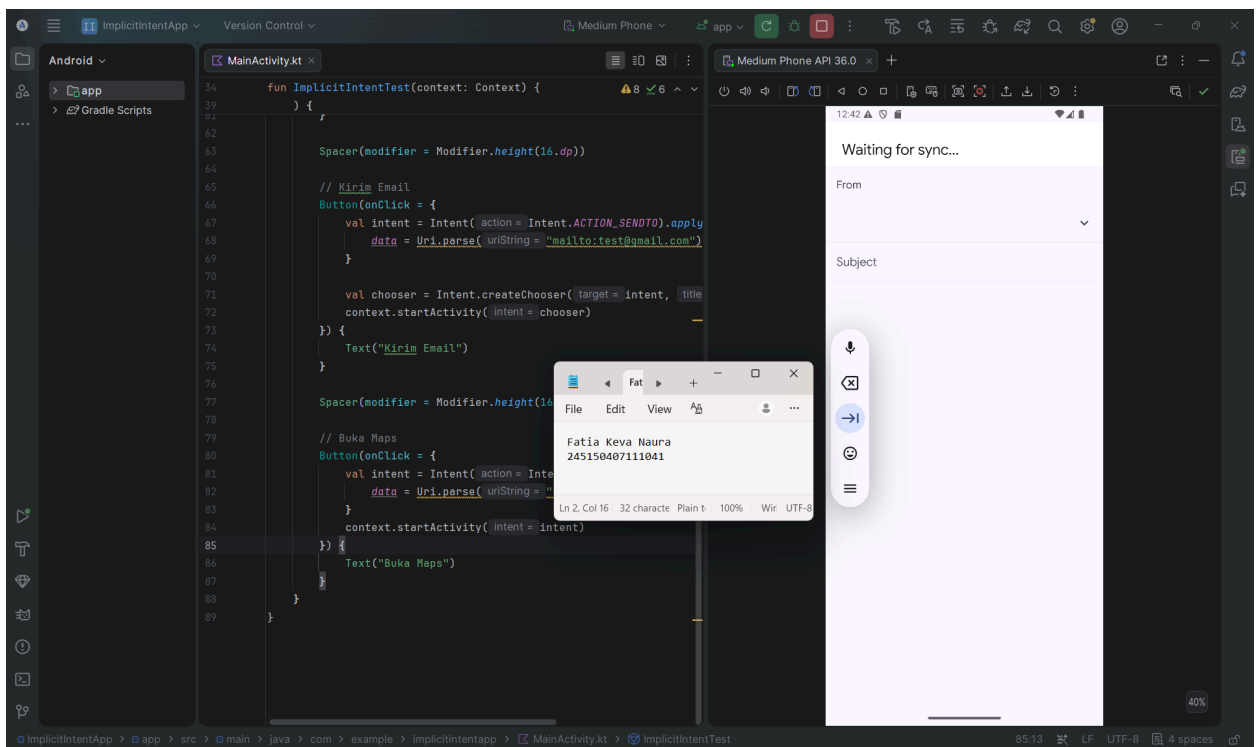
2. Klik Buka Browser



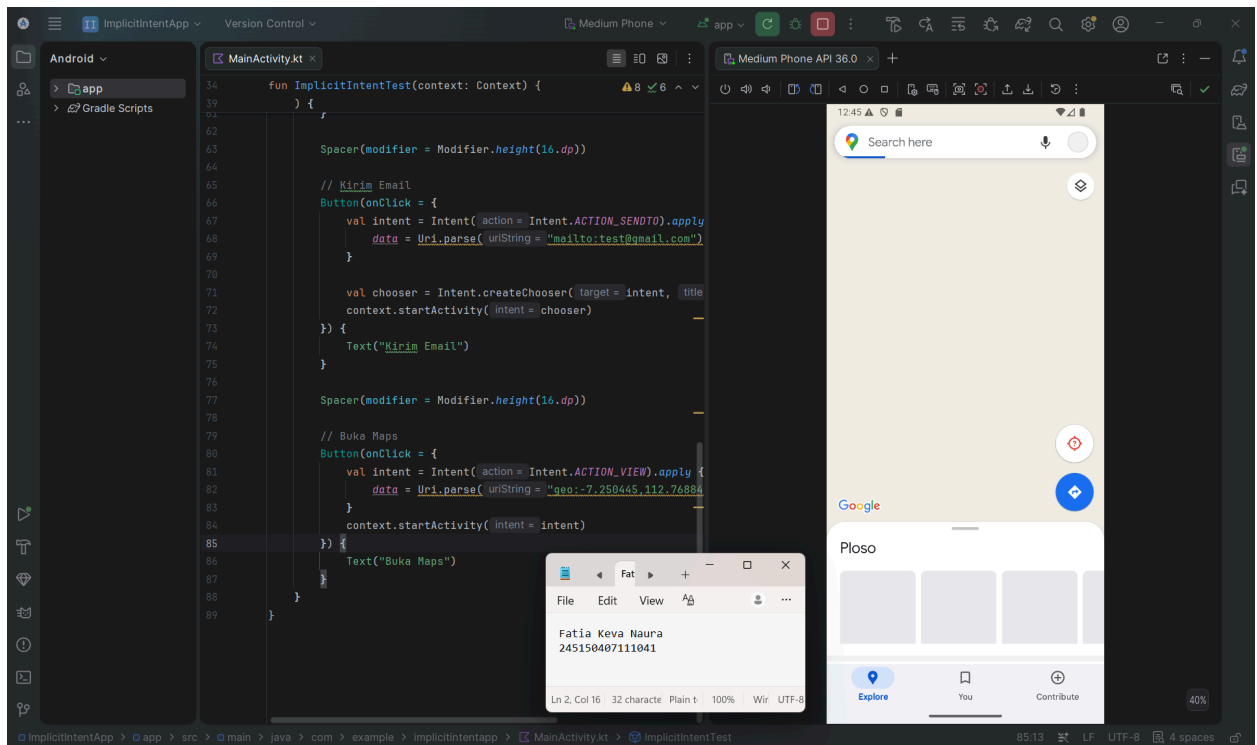
3. Klik Buka Telepon



4. Klik Kirim Email



5. Klik Buka Maps



6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya = Sudah terjawab diatas

7. Pada kode yang digunakan, terdapat beberapa action seperti:

Intent.ACTION_VIEW

Intent.ACTION_DIAL

Intent.ACTION_SENDTO

Apa perbedaan fungsi dari masing-masing action intent tersebut?

= Intent.ACTION_VIEW, Intent.ACTION_DIAL, dan Intent.ACTION_SENDTO memiliki fungsi yang berbeda.

- Intent.ACTION_VIEW digunakan untuk membuka atau menampilkan suatu data menggunakan aplikasi yang sesuai. Pada percobaan ini digunakan untuk membuka Google melalui browser dan membuka lokasi melalui Maps.
- Intent.ACTION_DIAL digunakan untuk membuka aplikasi telepon/dialer dengan nomor yang sudah ditentukan.
- Intent.ACTION_SENDTO digunakan untuk membuka aplikasi yang sesuai untuk mengirim pesan, pada percobaan ini digunakan untuk membuka aplikasi email dengan alamat tujuan tertentu.

Jadi, perbedaannya terletak pada tindakan yang ingin dilakukan oleh Intent.

12. Pada kode yang digunakan, terdapat beberapa data seperti:

```
data = Uri.parse("mailto:test@gmail.com")
data = Uri.parse("https://www.google.com")
data = Uri.parse("geo:-7.250445,112.768845")
```

Apa fungsi data pada intent ?

= data pada Intent berfungsi untuk memberikan informasi atau tujuan yang akan diproses oleh action Intent. Data dituliskan menggunakan Uri.parse() dan disesuaikan dengan jenis action yang digunakan.

Contohnya:

- data = Uri.parse("<mailto:test@gmail.com>") = digunakan untuk memberikan alamat email tujuan.
- data = Uri.parse("<https://www.google.com>") = digunakan untuk memberikan alamat website yang akan dibuka.
- data = Uri.parse("geo:-7.250445,112.768845") = digunakan untuk memberikan koordinat lokasi yang akan dibuka pada Maps.

Jadi, data memberikan informasi yang diperlukan oleh sistem Android untuk menjalankan action pada Intent.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

= Action dan Data saling berhubungan dalam menentukan aplikasi yang sesuai untuk menjalankan Intent. Action menunjukkan tindakan yang ingin dilakukan, sedangkan Data menunjukkan data atau tujuan yang akan diproses. Sistem Android kemudian mencocokkan keduanya dengan aplikasi yang dapat menangani Intent tersebut.

Contohnya:

- ACTION_VIEW + <https://www.google.com> -> sistem mencari aplikasi browser.
- ACTION_DIAL + nomor telepon -> sistem mencari aplikasi telepon/dialer.
- ACTION_SENDTO + <mailto:test@gmail.com> -> sistem mencari aplikasi email.
- ACTION_VIEW + geo:... -> sistem mencari aplikasi Maps/peta.

Jadi, Action menentukan “mau melakukan apa”, sedangkan Data menentukan “data apa yang digunakan atau dituju”.